

启发式搜索

经过前面我们的学习之后，我们会发现，很多时候搜索就是一种暴力的办法，特别是深度优先搜索，但是暴力的方法存在一个问题，那就是效率很低，很多时候不能满足我们的需要。所以后来的人就在基本的深搜和广搜的基础上研究出了很多种优化的搜索办法。

今天我们主要介绍一种高效的搜索方法——**启发式搜索**
什么是启发式搜索呢？

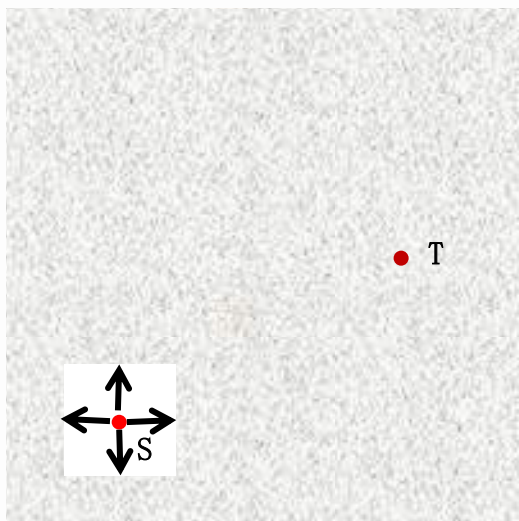
和普通的搜索相比，他不但要关注已经走过的路径，而且还要对未来的可能路径做一个预估。

对于一条完整路径，我们可以把它分成两部分，第一部分是已经走过的部分，这部分的答案是已知的，还有一部分是还未走过的，肯定是未知的，所以我们只能预估。

启发式搜索

我们先思考一下深搜和广搜慢的原因：对于每一个位置，有多个方向可以选择，我们一般都是给定固定的选择顺序，也就是随机性的搜索下去，这样的话，就很容易给出卡算法的数据，那能不能根据上面的搜索做一个优化呢？

很容易想到一种优化方法，并且也是我们在生活中经常用到的一种办法，只是我们没注意而已。



我们想象一下如果身处一个迷宫之中，我们知道终点在你的右上角，那么此时你面前有上下左右四条路可以选择，那么你会先选择哪一条呢？很明显，我们会主观的先选择向上或者向右的那一条。

启发式搜索

这就是普通搜索的一个简单的优化，但是实际上没啥用，因为完全可以出几组数据让往左上角最终走到死胡同，而往右下角迂回才是正解。这样效率并没有太大的变化。

前面这种搜索我们可以理解为贪心式搜索，每次只考虑当前最优，即局部最优，但是和贪心一样，局部最优并不是整体最优，所以我们还要考虑一下后面的情况。

但是后面的路径我们都还没有尝试过，肯定是没有办法知道最短路径的，如果知道，还需要我们写代码干嘛，所以这就需要我们z对后面的路径长度做一个估计，即写一个**估价函数**来估计后面的路径，这就是启发式搜索的核心，一个估价函数的好坏决定了一个程序的效率。

估价函数

关键是估价函数怎么构造呢？有什么限制吗？既然后面的路径我都不知道距离，该如何估计呢？

我们先考虑 $n*n$ 的方格的深搜。已知起点为 s ，终点为 t ，此时我已经到达了点 p ，并且此前我得到的最近距离为 $\min$ 。那么我应该如何判断是否该继续走下去呢？

很容易想到 $L_{sp} \geq \min$ ，就返回，如果我们加上启发式搜索的思想呢？构造一个估价函数呢？

当前已经确定的路径长度为 L_{sp} ，而 L_{pt} 的长度是未知的，但是我们可以估计一下 G_{pt} ，只要 $L_{sp} + G_{pt} \geq \min$ ，我们就认定通过该点到达终点的距离不可能是最短距离，那么我们就可以提前结束搜索了。

那么首先我们先思考一下？估计的距离和真实的距离的关系？是一定要大于，还是一定要小于，还是随意指定。

估价函数

首先，想都不需要想，随意指定是肯定不可能的，比如我指定所有点到终点的估计距离都 k ，那么对搜索的效率而言是没有任何意义的，当然，一定相等也是不可能的，既然我都知道每个点到终点的真实距离了，还需要找最短路径干嘛。

我们也很容易想到，估计路径 $>$ 实际路径也是错误的，想一个极端的例子，我让所有的估计距离都为正无穷，那么肯定满足条件，但是呢，对搜索效率来说是没有任何帮助的。再想一下，如果 $G_{pt} > L_{pt}$ ，那么很有可能 $L_{sp} + L_{pt} < \min$ 但是 $L_{sp} + G_{pt} > \min$ ，那这样不但没有提高效率反而你的程序都是错误的了。所以估价函数必须满足的要求是：

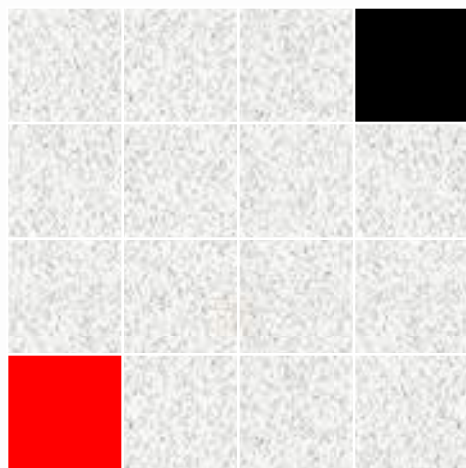
对于任意一个点：估计距离 $\leq$ 实际距离

并且估计距离越接近实际距离，效率越高

估价函数

我们思考一下，当估计距离=实际距离的时候，那么我们是不是可以不做任何无用功的直接一条路径搜索到从起点到终点的最短路径。

那我们再思考一下，对于前面说的 $n*n$ 的迷宫的深搜，我们如何构造估价函数呢？才能保证估计距离 $\leq$ 实际距离。



估价函数

是不是很容易想到，就是曼哈顿距离。

曼哈顿距离： $|x_1 - x_2| + |y_1 - y_2|$

也就是从一个点到另一个点，如果只能上下左右四个方向走，中间要经过的最短步数。

也就是说，如果当我到达点p的时候，如果后面没有任何障碍，后面每次都沿着最短路径走，到达终点的步数都 $\geq \min$ 的时候，那么是不是就可以说明这条路径肯定就不是最短路径了。这是一个很明显的道理。

当然，并不是说，每次搜索的时候，估价函数都是曼哈顿距离，这里只是举一个例子，只要你能保证估计距离 $\leq$ 实际距离，那么你的估价函数就是正确的，并且两者越接近，那么效率就越高。



搜索的优化

经过前面的学习，这也给我们做搜索题的时候提供了一种思路，我们不但要考虑当前最优状态，还需要对到达当前状态之后的剩余状态做一个估计，根据估价函数的设计原理，我们可以理解为，当到达当前状态时，如果后面每次我都通过理论上最优的办法去到达最终状态，如果效率比当前的最优解都还要差的话，那么这种状态就没有必要继续搜索下去了，这样就大大的节省了搜索的时间，提高了搜索的效率。



例题

给定一个1到n的排列 x ，每次你可以将 $x_1 \cdots x_i$ 翻转，问你需要多少次操作才能将该需要变为升序序列。 $N \leq 25$

例题

该题就是一道简单的搜索题，只要难度在于剪枝上面，我们要善于发现规律，对于每一次翻转，最多只能改变相邻数字发生改变的只有第一个数和第 i 个数，当第 i 个数与相邻的后一个数之差刚好为1时，这次交换导致求解的期望次数一定会+1.

交换之后，相邻的两个数之间的差如果 >1 ，那么肯定至少还需要一次交换才可以达到最终的效果，所以我们完全可以用启发式搜索，当前操作步数+后序估计步数(当前序列中 >1 的相邻数的对数) $>$ 当前最优答案

启发式搜索

启发式搜索这种思考方式可以用于我们很多写爆搜的题目中，在搜索过程中，对从当前状态到最终状态的最优方案做一个预估，那么你的搜索效率就会达到事半功倍的效果。

启发式搜索不但可以用于爆搜中，还可以用在求解k短路题目中。我们以前学习的图论算法都是求解最短路的，那么我们有没有想过，如果不是求解最短路，而是求解次短路（路径可以重复），那么我们应该如何思考呢？

梦境迷宫

【题目描述】

亮亮进入了一座迷宫。这座迷宫叫做“梦境迷宫”，亮亮只有走出这座迷宫，才能从睡梦中醒来。梦境迷宫可以用无向图来表示。它共有 n 个点和 m 条双向道路，每条道路都有边权，表示通过这条道路所需的时间，且每条道路可以多次经过。亮亮位于一号点，而出口则是 n 号点。原本，亮亮该找到一条最短路，快速冲出迷宫，然而，梦境迷宫的特殊之处在于，如果沿着最短路到达出口，亮亮就会永远陷入梦境。因此，亮亮必须寻找一条次短路。次短路的长度须严格大于最短路（可以有多条）的长度，同时又不大于所有除最短路外的道路的长度。

你的任务，就是编写一个程序，帮助亮亮找到通向出口的次短路。

【输入格式】

第一行有两个整数 n 、 m ，表示迷宫内共有 n 个点， m 条边。

接下来 m 行，每行三个整数 x 、 y 、 z ，表示结点 x 和 y 之间连有一条边权为 z 的无向边。

【输出格式】

一个整数，表示次短路的长度。

【样例输入】

```
4 4
1 2 2
2 4 4
2 3 3
3 4 4
```

【样例输出】

```
9
```

【样例解释】

最短路：1 -> 2 -> 4 (长度为 $2+4=6$)

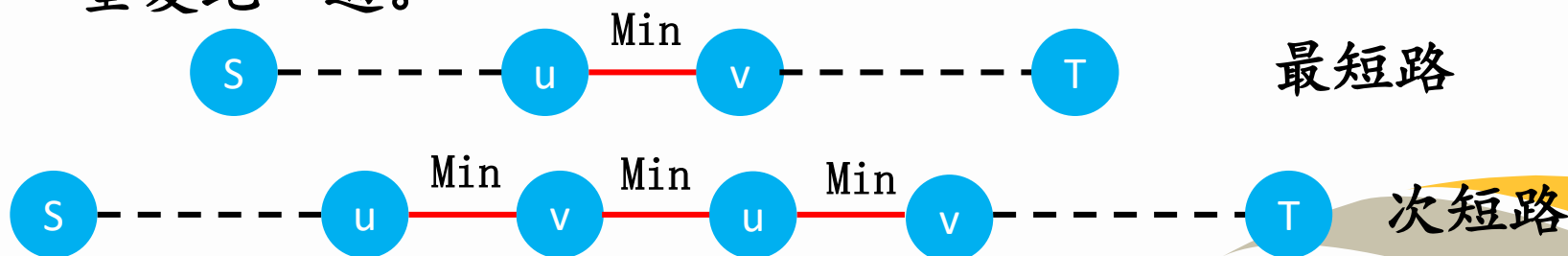
次短路：1 -> 2 -> 3 -> 4 (长度为 $2+3+4=9$)

解题思路

图的最短路我们都会求解，直接一个dijkstra或者SPFA就可以搞定了，但是该题是求解次短路，并不是最短路，应该如何来做呢？

当我们是在没有思路的时候，我们可以考虑搜索来做。但是我们以前做的搜索本质还是求解最短路，那么我们该如何剪枝呢？

既然没有办法直接用搜索求解最短路，那么我们不如假设一条可能的次短路，用哪一条最接近并且可能就是次短路呢？对于起点到终点的最短路上的路径最短的那一条路重复跑一遍。



解题思路

再得到假定的次短路的长度 L 之后，我们就可以搜索了：

```
if (step ≥ L) return ;
```

但是这样搜索效率特别低，因为深搜本来效率就低，而且该题路径还可以重复，虽然我们很容易想到次短路路径最多重复一次，但是效果比普通深搜还低。如果要效率高一点，那么我们思考下是否可以构造出一个估价函数，根据估计距离 $\leq$ 实际距离，对于任意一个点 u ，到终点 T 的真实距离 $\geq u$ 到 T 的最短路距离，那么我们完全可以用前面的最短路算法中得到的任意一个点到终点的最短路来做估价函数。所以我们在求解最短路的时候应该求解的是终点到其他点的最短路，而不是起点到任意点的最短路。

解题思路

```
void dfs(int id,int step)
{
    if(step+dis[id]>=s)//s为假定的次短路
        return ;
    if(id==n&&step>dis[1])//相同路径不算
    {
        s=step;
        return ;
    }
    for(int i=head[id];i;i=edge[i].next)
    {
        int v=edge[i].to;
        dfs(v,step+edge[i].w);//不能标记
    }
}
```


K 短路

这是启发式搜索在次短路中的应用，但是如果是k短路呢？比如 $k \leq 1000$ ，那么我们又改如何求解呢？

还是和刚才一样，先估计一个k短路出来？？？我们发现完全没有办法知道到底哪一条路可能是k短路。只能往大了估计，差不多就是正无穷了或者很大很大了，根本没有参考价值。

给定一个N个点，M条边的有向图，求从起点S到终点T的第K短路的长度，路径允许重复经过点或边，其中 $S, T \leq 1000, M \leq 100000, K \leq 1000$ 。

P0J 2449

K 短路

如果我们对dijkstra算法理解得比较透彻的话，我们会发现dijkstra算法不仅仅可以求解最短路，完全可以直接求解k短路。

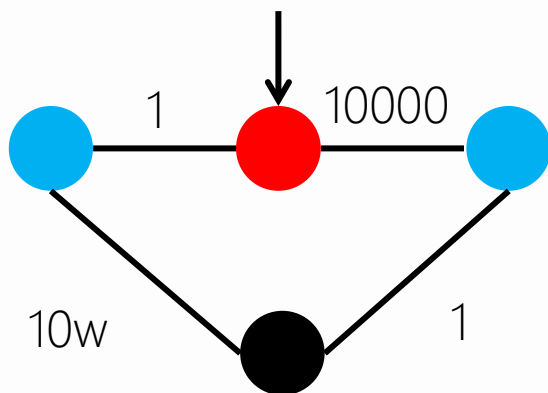
如果实在是想不到，我们不妨先思考一下在我们用广搜求解最短路的时候，是否可以用广搜求解k短路。

对于广搜来说，第一次搜到一个点就是到该点的最短路，那我们思考过，如果拓展到终点之后不结束，继续拓展呢？也就是说，当我们搜索到一个点放入队列中的时候，我们标记这个点，当他被拓展出队的时候，我们就取消标记，那么任意一个点就会被多次拓展，那么第一次拓展到该点时最短路，那么第二次拓展到呢？第三次拓展呢？

K 短路

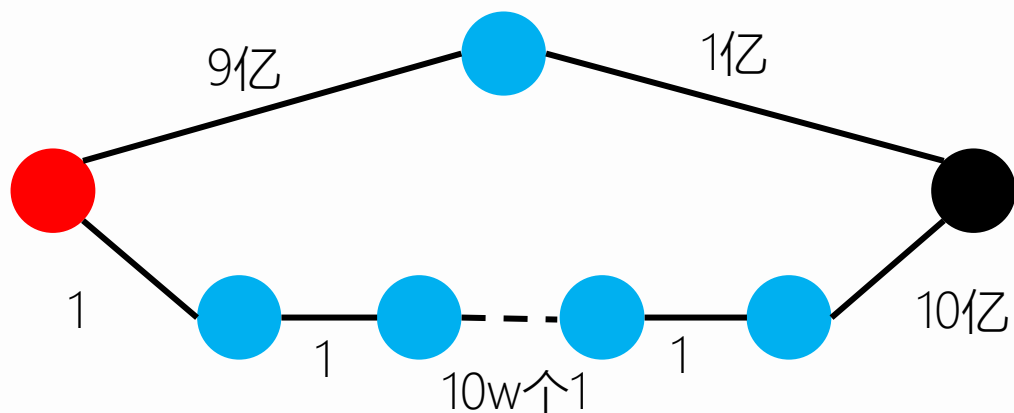
所以我们的dijkstra算法也是一样，当我们第一次拓展到该点时，得到的就是起点到该点的最短路，当一个点出队之后就取消标记，表示该点可以再次被访问到，那么当这个点被标记k次的时候，得到的就是起点到该点的k短路。所以我们很容易想到第一种解法：直接跑dijkstra，当终点第k次出队的时候，路径的长度就是最终的答案。

但是这样做有一个很大的问题。



K 短路

因为我们每次都是选择当前最优的点，并且一个点可以多次重复选择，那么最终就会导致，权值为1的点会被选择10000次之后才会去拓展权值为10000的点，可是万一权值为10000的点后面马上就是终点呢？那这样岂不是效率特别低？一直在走错误的道路。最坏时间复杂度为 $K * M \log M$ 。有的同学可能会说，优化一下不就好了，如果一个点出队了 k 次之后那么就完全没必要管他了，因为肯定不是 k 短路。好，我们来看一个极端的例子。



下面的一排点都会被拓展 k 次之后再拓展下一个点，效率超低！！！！

A* 算法

这就要用到我们的A*算法。如果我们考虑启发式搜索应该如何写估价函数呢？我们要求的是k短路，那么其中的某些边会被经过不止一次，那么对于到达的当前的每一个点 p ，我们知道 L_{sp} ，而我们需要估计的就是 G_{pt} ，并且估计值肯定小于等于该点到终点的k短路径中的距离。很容易想到，估价函数就写终点到任意一个点的最短路径就可以了。这就是一个优秀的估价函数。

接下来就是k短路的具体实现过程。

第一步：确定估价函数——最短路。

即求出终点到任意一个点的最短路径的距离，因为是有向图，所以我们要先建反向边，从终点开始跑最短路。得到一个dis数组。

A*算法

第二步：确定估价函数——最短路。

跑A*算法。对于一个当前点 p ，记录 L_{sp} 的距离(起点到 p 点的实际距离)，记录 G_{pt} 的距离($dis[p]$)，每次从队列中取出 $L_{sp}+G_{pt}$ 最小的一个优先拓展，然后更新 L 和 G 的答案。

注意：

为了避免被卡死，任意一个点不能被拓展超过 k 次，假设一个点被拓展超过 k 次，那么这条路径就一定不是 k 短路，因为当前已经被拓展超过 k 次了。

这道题还有一个坑点是，如果起点=终点，不走是不算一条路径的。

A* 算法

```
Node tmp;
tmp.id=s;
tmp.h=0;
tmp.g=dis[s];
que.push(tmp);
memset(cnt,0,sizeof(cnt));
while(!que.empty())
{
    Node tmp=que.top();
    que.pop();
    cnt[tmp.id]++;
    if(cnt[tmp.id]>K)
        continue;
    if(cnt[t]==K)
        return tmp.h; // 终点找到了最短路
    for(int i=head[tmp.id];i;i=edge[i].next)
    {
        int v=edge[i].to;
        Node tt;
        tt.id=v;
        tt.h=tmp.h+edge[i].w;
        tt.g=dis[v];
        que.push(tt);
    }
}
```

```
struct Node
{
    int id;
    int h; // 起点到当前点的已知最短距离
    int g; // 终点到当前点的最短距离
    friend bool operator <(Node a, Node b)
    {
        return a.g+a.h>b.g+b.h;
    }
};
priority_queue<Node> que;
```



必做练习题

POJ 2449: Remmarguts' Date

MSOJ 1339: 梦境迷宫

洛谷 2901: 牛慢跑



迭代加深搜索

我们知道，深搜的用途比较广，比较节省空间，但是效率比较低；广搜虽然效率比较高，但是耗费空间比较大，用途也稍微窄一点。那么有没有一种搜索能够结合深搜和广搜的优点呢？

后来人们在研究中，发明了一种新的搜索方法——迭代加深搜索。他在很多时候会有意想不到的效果。

迭代加深搜索其实很简单，就是DFS+BFS，他先假设答案就在第 k 层，那么从起点开始以深搜的方式搜索第 k 层的所有点，如果没有搜到，那么就假设答案在第 $k+1$ 层，再以深搜的方式去搜索去 $k+1$ 层的所有点。一直循环下去，直到有解为止。

迭代加深搜索又可以理解为按层拓展的深度优先搜索。

迭代加深搜索

```
int k=1;//先从第一层开始搜索
while(1)
{
    dfs(1,k);//假设答案就在第k层，那么搜到第k层就返回
    if(flag==0)//搜完第k层都没有搜到答案
        k++;
    else//答案就在第k层，跳出循环
        break;
}
```

POJ 3134

题目大意：给定一个数 x ，问如何通过乘除法变为 x^n . 最少次数是多少？每次可以乘以或除以已经被构造出来的数。

比如 $x^2 = x * x$ (1次)

$x^3 = x^2 * x$ (2次)

$x^4 = x^2 * x^2$ (2次)

$x^5 = x^2 * x^3$ (3次)

x^6 (3次)

x^7 (4次)

x^{29} (7次)

x^{30} (6次)

x^{31} (6次)

解题思路

该题因为有多组数据，直接用深搜，那么剪枝只能用 $n-1$ 来剪枝(每次都乘以 x ，最多 $n-1$ 次就可以变为 x^n)。但是数很多的情况下，随机数据很多时候根本用不到 $n-1$ 次，特别是后期 n 比较大的时候，真实次数远小于 $n-1$ 。所以直接深搜效率很多，所以我们可以考虑迭代加深搜索。

```
num[0]=1; // 记录所有出现过的次数
int k=1;
while(1)
{
    if(dfs(0,1,k))
        break;
    k++;
}
printf("%d\n",k);
```

解题思路

```
bool dfs(int cur,int x,int dep)//操作次数, 最大指数
{
    if(num[cur]==n) return 1;
    if(cur>=dep) return 0;
    x=max(x,num[cur]);
    for(int i=cur;i>=0;i--)
    {
        num[cur+1]=num[cur]+num[i];//乘法
        if(dfs(cur+1,x,dep))
            return 1;
        if(num[cur]>num[i])
            num[cur+1]=num[cur]-num[i];//除法
        else
            num[cur+1]=num[i]-num[cur];//除法
        if(dfs(cur+1,x,dep))
            return 1;
    }
}
```

解题思路

但是这样做效率还是很低，虽然比普通的深搜效果好一些，所以我们在使用迭代加深搜索的时候还是要考虑启发式搜索的思想。考虑一下当前状态到最终状态最少需要多少步？

这个很容易想到，对于前面出现的最大次数，每次平方，最少需要平方多少次。就是最快的方法。

所以要加上这个剪枝。

```
if(x*(1<<(dep-cur))<n)//如果最快增长在第dep层都到不了目标  
return 0;
```

POJ 3460 Booksort

【例题】Booksort^{POJ3460}

给定 n ($1 \leq n \leq 15$) 本书，编号为 $1 \sim n$ 。在初始状态下，书是任意排列的。在每一次操作中，可以抽取其中连续的一段，再把这段插入到其他某个位置。我们的目标状态是把书按照 $1 \sim n$ 的顺序依次排列，求最少需要多少次操作。若操作次数 ≥ 5 ，则直接输出字符串“5 or more”即可。

解题思路

很容易看出这是一道搜索题，我们抽取长度为 i 的一堆书，取出来之后再放进去，放入的位置做最多有 $n-i+1$ 种，但是我们会发现每种方案都重复了一次。比如由1324，变为1234，即可以3后移，也可以2前移。所以中的方案数为 $i*(n-i+1)/2$ ($1 \leq i \leq n$)，当 $n \leq 15$ 时，每一次移动大概有560种可能。因为题目中要求操作次数不能大于等于5次，所以最多 560^4 。很明显会超时。

那我们该需要优化呢？第一种是双向bfs，从起点和终点同时向中间状态搜索2步，时间复杂度为 $2*560^2$ 。时间复杂度是可以接受的。

第二种是ID A*，也就是限制深度的迭代加深搜索。这种方法代码比较短，但是我们要思考如何才能估计后面的操作次数。

解题思路

和前面的A*一样，ID A*的估价函数也必须要满足前面所学习的估价函数的要求，也就是估计步数 $\geq$ 实际步数，否则估计就是错误的，并且越接近越好。

那么对于当前状态，我们要如何判断最少需要多少步才可能到达最终状态呢？

我们可以思考一下，一次操作最多可以改变多少本书的后继，假设这些操作都是正确的，也就是说每次改变刚好都把错误的后继变正确了。



如果我们移动一堆书，我们会发现改变了三个区域的后继：这堆书前面那本书的后继改变了；这堆书中最后一本书的后继改变了，插入的位置前的最后一本书的后继改变了。

解题思路

也就是说，当我们移动一堆书的时候，最多可以改变三本书的后继，也就是最多可以让三个错误的后继变为正确。那么启发式函数就好写了：

当前搜索层数+改变剩下的错误后继的最少次数>假设答案所在的层数。

解题思路

第一步：统计当前状态错误后继的个数。

```
if(a[1]!=1)
    wa++;
for(int i=1;i<n;i++)
{
    if(a[i+1]!=a[i]+1)
        wa++;
}
```

第二步：确定当前状态最少需要的次数。

```
int num=(wa+2)/3;//表示最少次数，wa/3向上取整
```

解题思路

第三步：双重循环枚举取出哪一堆书。再枚举放入到哪个位置

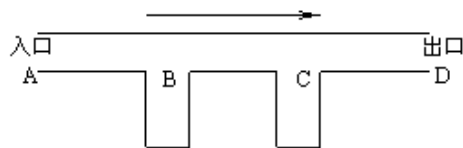
```
for(int i=1;i<=n;i++){
    for(int j=i;j<=n;j++)//取出[i,j]之间的书
    {
        for(int k=1;k<=n+1;k++)//可以插入的位置有n+1个，去除区间[i,j+1]
        {
            if(k>=i&& k<=j+1) continue;
            if(k<i)
                change(k,i-1,i,j);//相当于交换区间[l,i-1]和[i,j]的位置
            else if(k>j+1)
                change(i,j,j+1,k-1);//相当于交换区间[i,j]和[j+1,k-1]
            递归回溯
        }
    }
}
```

解题思路

题目描述

展开

如图所示 l ，某火车站有 B, C 两个调度站，左边入口 A 处有 n 辆火车等待进站(从左到右以 a, b, c, d 编号)，右边是出口 D ，规定在这一段，火车从 A 进入经过 B, C 只能从左向右单向开，并且 B, C 调度站不限定所能停放的车辆数。



从文件输入 n 及 n 个小写字母的一个排列，该排列表示火车在出口 D 处形成的从左到右的火车编号序列。输出一系列操作过程，每一行形如“ h, L, R ”的字母序列，其中 h 为火车编号， L 为 h 车原先所在位置(位置都以 A, B, C, D 表示)， R 为新位置。或者输出‘NO’表示不能完成这样的调度。

输入格式

一个数 $n(1 < n < 27)$ 及由 n 个小写字母组成的字符串。

输出格式

可以调度则输出最短的调度序列，当有多种方案时输出按操作(L, R)字典序最小的方案，不可以调度时则输出‘NO’。

解题思路

该题看上很难，实际上也很难，一道题不会做就想暴力怎么做。我们先思考一下每次挪动有哪几种可能，实际上就六种可能，A-B, A-C, A-D, B-C, B-D, C-D, 那么每次挪动肯定是这六种中的一种，而且既然要最小字典序最小，那么操作肯定是有顺序的。

我们既然要知道每次挪动的具体情况，那么我们只需要开四个栈就可以了，分别表示A, B, C, D, 因为只能从一头进出，那么就完全可以用栈模拟。

该题，我们考虑一下启发式搜索，既然每次都只可以挪动一个，那么D栈里面的盘子数量就可以用于启发式搜索。同样，该题可以用迭代加深搜索，这样第一次搜到的答案就是正解。以免操作错误，已知搜索下去浪费时间。

解题思路

该题看上很难，实际上也很难，一道题不会做就想暴力怎么做。我们先思考一下每次挪动有哪几种可能，实际上就六种可能，A-B, A-C, A-D, B-C, B-D, C-D, 那么每次挪动肯定是这六种中的一种，而且既然要最小字典序最小，那么操作肯定是有顺序的。

我们既然要知道每次挪动的具体情况，那么我们只需要开四个栈就可以了，分别表示A, B, C, D, 因为只能从一头进出，那么就完全可以用栈模拟。

该题，我们考虑一下启发式搜索，既然每次都只可以挪动一个，那么D栈里面的盘子数量就可以用于启发式搜索。同样，该题可以用迭代加深搜索，这样第一次搜到的答案就是正解。以免操作错误，已知搜索下去浪费时间。

```
for(int i=1;i<=3*n;i++)//最多3*n层  
dfs(1,i); //ID-A*搜索
```

解题思路

```
if(s4[tp4]!=s[tp4])  
return ;//如果已经出栈的部分顺序和目标顺序不一样，那么直接返回  
if(step-1+(n-tp4)>num)//每一步最多出栈一个，如果每次都出栈还搜不到终点
```

```
if(tp1)//A--B  
{  
    l[step]='A';  
    r[step]='B';  
    h[step]=s1[tp1--];  
    s2[++tp2]=h[step];  
    dfs(step+1,num);  
    s1[++tp1]=h[step];  
    tp2--;  
}
```

```
if(tp3)//C--D  
{  
    l[step]='C';  
    r[step]='D';  
    h[step]=s3[tp3--];  
    s4[++tp4]=h[step];  
    dfs(step+1,num);  
    s3[++tp3]=h[step];  
    tp4--;  
}
```


练习题

A*:

P0J 2449:Remmarguts' Date

P0J 1945:Power Hungry Cow

P0J 1077 :八数码

MS0J 1339 : 梦境迷宫

洛谷 2483 : 魔法猪学院

ID A*:

P0J 3460 Booksort

P0J 2331 Water pipe

P0J 4007 Flood-it!

洛谷 2324 骑士精神

洛谷 1139 单向双轨道